Algorithm Analysis and Simulation Toolkit

FINAL PROJECT FOR ALGORITHMS & COMPLEXITY
Term 2, SY 2025 - 2026

Project Overview

Students will design and implement a software application that demonstrates and analyzes fundamental algorithms. The project focuses on three major components:

- Comparison of Sorting Algorithms
- Simulation of Minimum Spanning Tree Algorithms (Kruskal's and Prim's)
- Simulation of Recursive Functions

The application should allow users to input data, visualize algorithm behavior, and analyze performance.

Project Objectives

By completing this project, students will:

- Understand and implement common sorting algorithms
- Understand Minimum Spanning Tree (MST) algorithms
- Visualize recursion and recursive call behavior
- Compare algorithm efficiency using time complexity analysis
- Develop skills in algorithm visualization and simulation

System Requirements

The program must include:

- A menu-driven interface (CLI or GUI)
- Input validation
- Clear output and visualization
- Modular code structure
- Documentation and comments

Recommended languages:

- Python
- Java
- C++
- C#

Project Components

Part 1: Sorting Algorithm Comparison

Description

The system must implement multiple sorting algorithms and compare their performance on the same dataset.

Required Algorithms

Students must implement at least three of the sorting algorithms enumerated below:

- Bubble Sort
- Selection Sort
- Insertion Sort
- Merge Sort
- Quick Sort
- Random-Quick Sort
- Counting Sort
- Radix Sort

Features

The system must:

- Accept user input or randomly generate datasets
- Allow the user to choose dataset size
- Execute each sorting algorithm
- Display:
  - Sorted output
  - Execution time
  - Number of comparisons
  - Number of swaps (optional)

Output Example

```
Dataset Size:  1000

Algorithm        Time (ms)
Bubble Sort      45
Selection Sort   38
Insertion Sort   29
Merge Sort       5
Quick Sort       4
```

Optional (Bonus)

- Graph showing algorithm performance
- Step-by-step visualization

Part 2: Simulation of Kruskal's and Prim's Algorithm

Description

The system must simulate the creation of a Minimum Spanning Tree (MST) using:

- Kruskal's Algorithm
- Prim's Algorithm

These algorithms are used in graph optimization problems.

Required Features

The system should:

- Accept graph input:
  - Number of vertices
  - Weighted edges
- Display the graph structure
- Simulate each step of the algorithm

Kruskal's Algorithm Simulation

Show:

- Edge sorting
- Edge selection
- Cycle detection
- MST formation

Prim's Algorithm Simulation

Show:

- Starting vertex
- Edge selection
- Growing MST

Output Example

```
Edges selected for MST:
1 -- 2 (weight 3)
2 -- 4 (weight 5)
4 -- 5 (weight 6)
Total MST Cost = 14
```

Optional (Bonus)

- Graph visualization
- Step-by-step animation

Part 3: Simulation of Recursive Functions

Description

Students must demonstrate how recursion works by simulating recursive calls.

Required Recursive Functions

Implement at least three recursive algorithms:

- Factorial
- Fibonacci
- Tower of Hanoi

Features

The system must:

- Display recursive calls
- Show base cases
- Show return values

Example Output

```
factorial(4)
 -> 4 * factorial(3)
 -> 3 * factorial(2)
 -> 2 * factorial(1)
 -> 1
Result = 24
```

Tower of Hanoi Example

```
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
```

Optional (Bonus)

- Recursion tree visualization
- Call stack simulation

System Design Requirements

Students must include:

1. Flowcharts for major algorithms

2. Program Structure - Use modular design

   Example modules:

   - `sorting_module`
   - `graph_module`
   - `recursion_module`
   - `main_program`

3. Documentation should include:

- Algorithm explanation
- Program instructions
- Sample outputs

Deliverables

Students must submit:

- Source Code
- Project Report (PDF)
- Screenshots of the program
- Algorithm explanation
- Test cases

Bonus Opportunities (+10)

Possible extra credit:

- Graphical visualization of algorithms
- Performance charts
- Interactive recursion tree
- Large dataset testing
- GUI application

Deadline

All projects must be compressed to a zip file and uploaded in NeoLMS by **07 April 2026 (Tuesday), 11:59PM**. If the project is done by group (of 2 or 3), only 1 representative will submit.

Grading Rubric

Criteria	Excellent	Good	Fair	Poor	Points
Sorting Algorithms Implementation	All algorithms correctly implemented with comparison metrics	Minor issues but mostly correct	Some algorithms incomplete	Major errors or missing	20
Sorting Comparison Analysis	Clear performance comparison and explanation	Basic comparison provided	Limited analysis	No comparison	10
Kruskal's Algorithm Simulation	Correct and clearly demonstrated	Minor errors	Partial implementation	Incorrect or missing	15
Prim's Algorithm Simulation	Correct and clearly demonstrated	Minor issues	Partial implementation	Incorrect or missing	15
Recursive Function Simulation	Clear recursion demonstration with outputs	Mostly correct	Limited recursion display	Incorrect or missing	15
User Interface / Usability	Easy to use, well organized	Functional interface	Basic interface	Hard to use	10
Code Quality	Clean, modular, well-commented	Mostly organized	Some structure issues	Poor coding practices	5
Documentation / Report	Clear explanations and diagrams	Adequate documentation	Minimal documentation	Missing or unclear	5
Creativity / Extra Features	Visualization or advanced features	Some enhancements	Minimal extras	None	5

Total: 100 Points